

Algos de tri

30 mai 2022 14:16

- Tri insertion $\Theta(n^2)$
- Tri fusion $\Theta(n \log n)$
- Quicksort $\Theta(n^2)$
- Tri comptage $\Theta(n + \max(T))$

Entrée: séquence d'objets $T = [t_1, t_2, \dots, t_n]$ comparables avec l'opérateur $\leq$

Sortie: permutation $T' = [t'_1, t'_2, \dots, t'_n]$ des elts de T telle que $t'_1 \leq t'_2 \leq \dots \leq t'_n$

Version avec clé-valeur

Entrée: seq. de paires $T = [(t_1, v_1), (t_2, v_2), \dots, (t_n, v_n)]$
clé valeur
↓ ↓
 comparables avec $\leq$ (les t_i)

Sortie: permunt. $T' = [(t'_1, v'_1), \dots, (t'_n, v'_n)]$ de T
 t.q. $t'_1 \leq t'_2 \leq \dots \leq t'_n$

BD: id row ORDER BY id → id row

2	a16, 4	1	b, 3
1	b, 3	2	a16, 4
3	c, 8	3	c, 8

Tri-insertion(T)

pour $j = 2 \dots n$
 $val = T[j]$
 $i = j - 1$

[1 2 5 3 2]

$i=0$ $i=1$ $i=2$ $j=3$ $val=1$

[1 1 1]

$j=2$ $j=3$

c

```

    pour j = 2 ... n
    val = T[j]
    i = j - 1
    tant que i > 0 et T[i] > val
    | T[i+1] = T[i]
    | i--
    T[i+1] = val
    return T
  
```

val = 1

[1 2 3 5 2]

i = 2 stop i = 3 j = 4
val = 3

[1 2 x 3 → 5 → 2]

i = 2 val 2 j = 5

[1 2 2 3 5]

j = 2 j = 3

Thm: Tri-insertion s'exécute en temps $O(n^2)$ dans le pire cas.

Preuve: l'algo fait au plus n itérations sur j .

À chaque itér., l'algo exécute un nb constant c d'instruction, sans compter la boucle interne.

De plus, la boucle sur i fait au plus n itérations par tour de j , et cette boucle interne fait un nb constant d d'instructions.

Le nb d'instructions est donc au plus

$$\underbrace{n \cdot c}_{j} \cdot \underbrace{n \cdot d}_{i} = c \cdot d \cdot n^2 \in O(n^2).$$

Thm: le temps de tri-insertion est $\Omega(n^2)$ dans le pire cas.

Preuve: prenons $T = [n, n-1, n-2, \dots, 2, 1]$

Preuve: prenons $T = [n, n-1, n-2, \dots, 2, 1]$

À chaque $j \in \{2, 3, \dots, n\}$, l'algo doit décaler $j-1$ valeurs sur cette entrée.

Le # de décalages faits est donc au moins

$$1 + 2 + 3 + \dots + n-1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \in \Omega(n^2)$$

$j=2 \quad j=3 \quad j=4 \quad j=n$

Corollaire: le temps de tri-insertion est $\Theta(n^2)$ dans le pire cas.

Note: sur entrée $[1, 2, \dots, n]$, temps est $\Theta(n)$.

En fait, le temps est $\Theta(n+k)$, où $k = \#$ paires "mal placées"

ex: $[2 \quad 1 \quad 4 \quad 5 \quad 3]$



Thm: Tri-insertion trie correctement.

• Recette pour montrer qu'un algo itératif est correct

Trouver et prouver une propriété qui:

- ① est vraie après chaque itération (induction sur # de tours)
- ② prouve que l'algo fonctionne

$\cdot \quad \tau \quad 1 \quad \quad n \quad 1 \quad \quad + \quad n \quad 1 \quad 1$

(2) prouve que l'algo fonctionne

- Technique de l'invariant de boucle.

On prouve la propriété suivante:

[Après chaque de la boucle principale (sur j), le sous-tableau $T[1..j]$ est trié.]

On prouve par induction sur j .

Cas de base: $j=2$. À prouver: après le tour, $T[1..2]$ est trié.

Si: $T[1] > T[2]$, alors $T[1]$ a été décalé et $\text{val} := T[2]$ a été placé à $T[1]$, et donc $T[1..2]$ sera trié.

Si: $T[1] \leq T[2]$, alors rien ne sera décalé et $T[1..2]$ sera trié. ✓

Induction: on a $j > 2$ et on suppose qu'au début de l'itération avec j , le sous-tableau $T[1..j-1]$ est trié (hyp).

À prouver: après le tour de boucle avec j , $T[1..j]$ est trié.

Considérons i sur lequel le tour est sorti de la boucle interne.

On a que $T[1..i]$ est intact et $T[i+1..j-1]$ a été décalé. De plus, i est le plus petit indice qui satisfait $T[i] \leq T[j]$.

Donc le tableau à la fin du tour est

$$T[1..i] + T[j] + T[i+1..j-1] \quad // + \text{concaténation}$$

Puisque $T[1..j-1]$ était trié, alors $\leftarrow$ hypothèse d'ind.

$T[1..i]$ est trié et $\leq T[j]$ et $T[i+1..j-1]$
est trié et $> T[j]$ et donc le sous-tableau
est trié.



Tri fusion

trifusion(T)

si $|T| \leq 1$, return T

A = trifusion($T[1.. \frac{n}{2}]$)

B = trifusion($T[\frac{n}{2}+1..n]$)

S = []

a = 1

b = 1

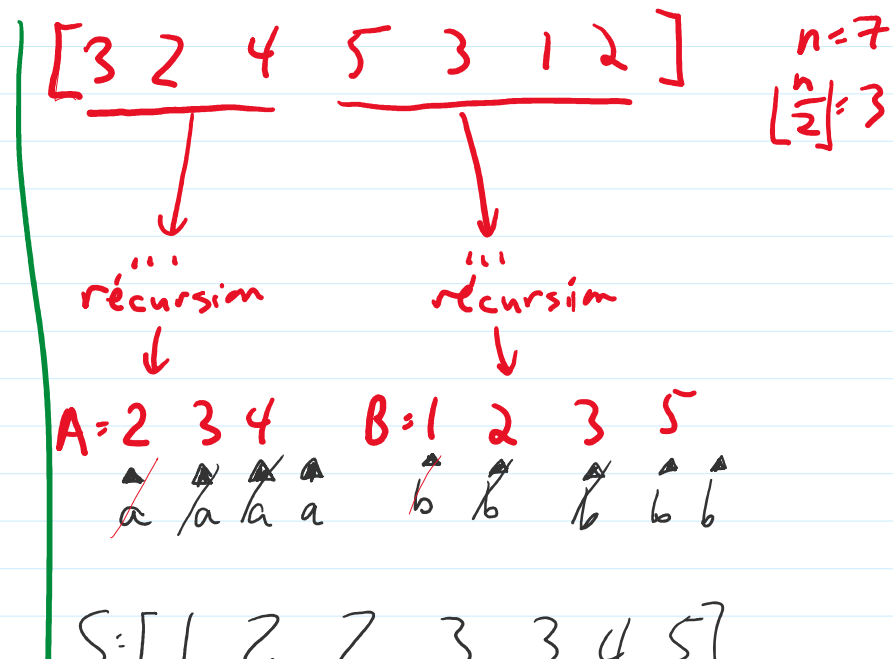
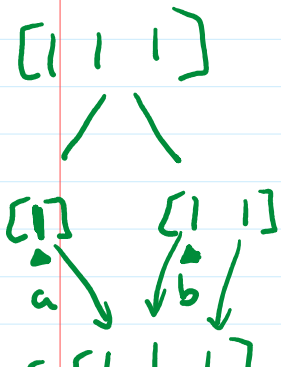
tant que a $\leq$ |A| et b $\leq$ |B|

si b > |B| ou a $\leq$ |A| et A[a] $\leq$ B[b]

| S.append(A[a])

a++

sinon



a ↓ ↓ b ↓
S = [1 1 1]

```

    | | append(a)
    | | a++
    | | sin(a)
    | | S.append(B[b])
    | | b++
    | | return S
  
```

S = [1 2 2 3 3 4 5]

Thm: le temps de trifusion est $\Theta(n \log n)$ dans le pire cas.

~~On~~ On a déjà argumenté que trifusion faisait toujours $n \log n$ comparaisons.

C-à-d, la complexité de trifusion ne dépend pas de l'entrée et est très proportionnelle à $\Theta(n \log n)$.

Quicksort

sur entrée T
Idée: choisir un elt pivot de T

A = les elts de T $\leq$ pivot

B = les elts de T $>$ pivot

trier A

[3 2 5 1 4]

└─┘
pivot = 3

A = [2, 1] B = [5, 4]

↓ réc.

↓ réc.

A = [1, 2] B = [4, 5]

trier A
trier B

return $A + [\text{pivot}] + B$ // + concaténation

$A = [1, 2]$ $B = [4, 5]$

return $[1, 2] + [3] + [4, 5]$

quicksort(T)

si $|T| \leq 1$, return T

pivot = T[1]

A = [], B = []

pour $i = 2 \dots n$

si $T[i] \leq \text{pivot}$

A.append(T[i])

sinon

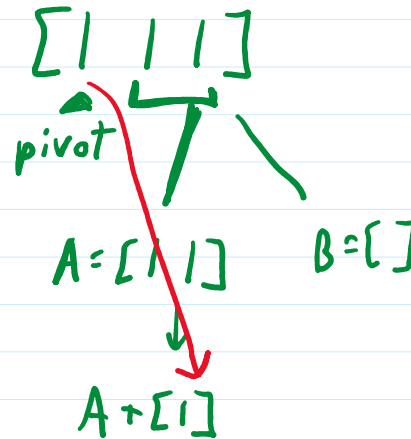
B.append(T[i])

A = quicksort(A)

B = quicksort(B)

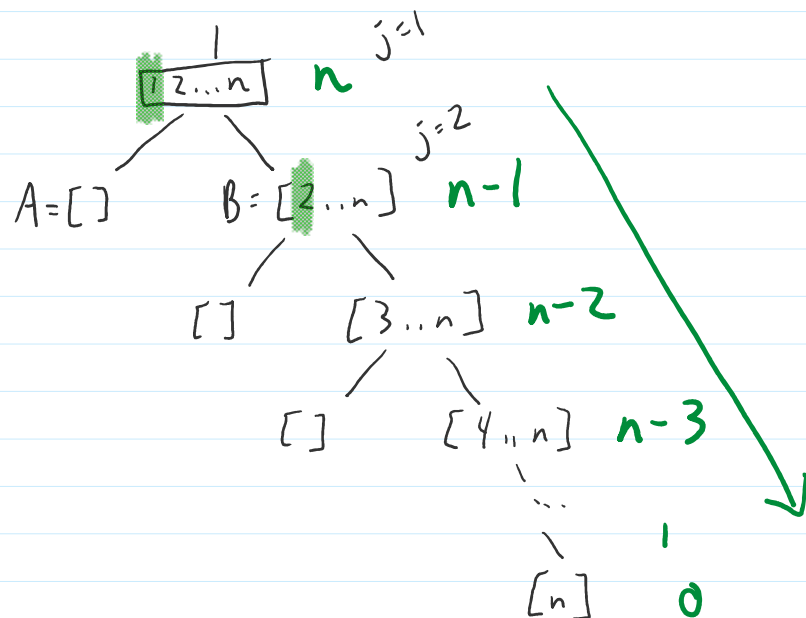
return $A + [\text{pivot}] + B$

n tours



Thm: le temps de quicksort est $\Omega(n^2)$ dans le pire cas.

Considérons $T = [1, 2, 3, \dots, n]$



Sur cette entrée,
l'algo fait un total
de n appels récursifs.
Pour chaque j , le j -ème
appel reçoit un tab
de taille $n-j+1$
et doit parcourir ce
tableau, pour bâtir $A+B$.

Le temps passé pour tous les appels est donc au moins

$$\sum_{j=1}^n (n-j+1) = \sum_{j=1}^n n - \sum_{j=1}^n j + \sum_{j=1}^n 1 = n^2 - \frac{n(n+1)}{2} + n$$

$$= n^2 - \frac{n^2}{2} - \frac{n}{2} + n$$

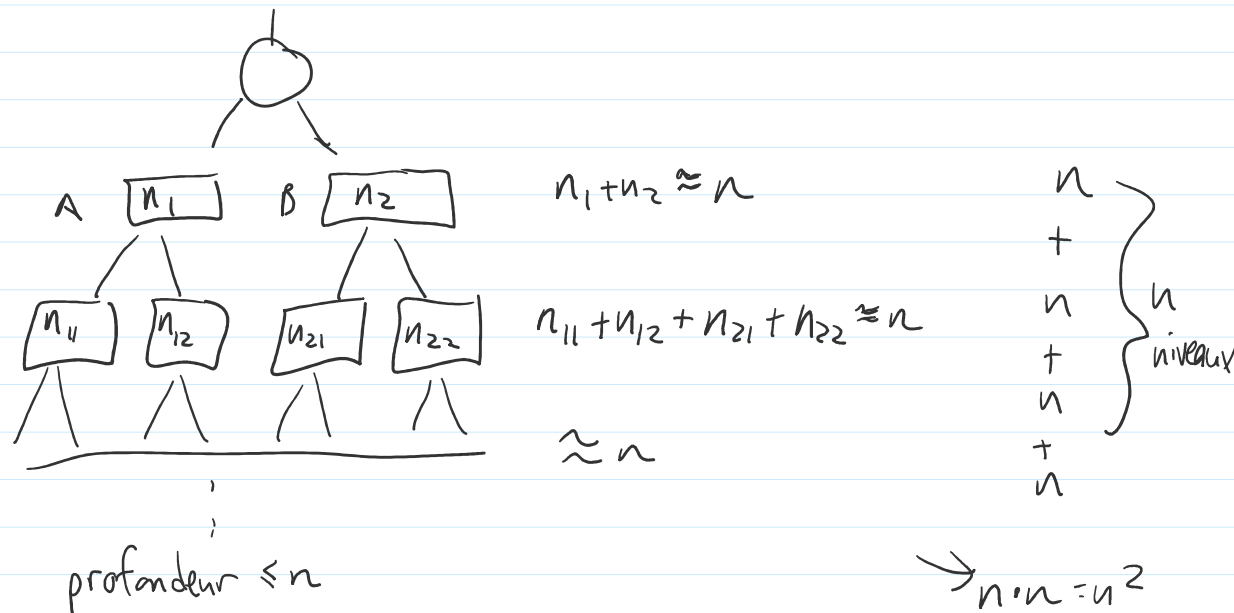
$$= \frac{n^2}{2} + \frac{n}{2} \in \Omega(n^2). \quad \square$$

$$= \frac{n^2}{2} + \frac{n}{2} \in \Omega(n^2). \quad \square$$

Thm: le temps de quicksort est $O(n^2)$.

Exercice.

Intuition:



Le meilleur algo de tri semble être $\Theta(n \log n)$.

En fait, c'est essentiellement optimal.

(tri par comparaison)

Thm: + + + ... non déterministe ...

(Tri par comparaison)

Thm: tout algo de tri qui base ses décisions selon le résultat de l'opérateur " $\leq$ " prend un temps $\Omega(n \log n)$.

Prochain cours: on peut faire mieux avec du tri sans comparaison.

Tri monceau

Monceau : insertion $\Theta(\log n)$
getMin $\Theta(1)$
deleteMin $\Theta(\log n)$

triMonceau(T)

M = new Monceau()

n fois — pour $i = 1..n$ // $n = |T|$

$O(\log n) = \sum_x M.insert(T[i])$

$S = T$

$\left] O(n \cdot \log n) \right]$

$S = []$

n fois - tant que $M.vide()$

$O(1)$ - $S.append(M.getMin())$
 $O(\log n)$ - $M.deleteMin()$
 $\times$ return S

$O(n \cdot (1 + \log n))$
 $= O(n + n \log n)$
 $= O(n \log n)$

$O(n \log n + n \log n) = O(n \log n)$

Tri sans comparaison

Entrée: paires clé-val $T = [(t_1, v_1), (t_2, v_2), \dots, (t_n, v_n)]$

telle que $\{t_1, t_2, \dots, t_n\} = \{1, 2, 3, \dots, n\}$

$T[1].clé = 2$ $T[2].clé = 1$

ex: $T = [(2, allo), (1, bonjour), (4, salut), (3, hola)]$

sortie triée: $[(1, bonjour), (2, allo), (3, hola), (4, salut)]$

bucket Sort Simplifié (T)

$O(n)$ $S = [nil] * n$ // tabl. de $n \times nil$ | $[\emptyset, \emptyset, \emptyset, \emptyset]$
 $S[2] = (2, allo)$

$O(n)$ $S = [\text{nil}] * n$ // tabl. de $n \times \text{nil}$

n fois pour $i = 1..n$

$O(1)$ $S[T[i].\text{clé}] = T[i]$

return S

$O(n)$

$[\emptyset, \emptyset, \emptyset, \emptyset]$

$S[2] = (2, \text{allo})$

$[\emptyset, (2, \text{allo}), \emptyset, \emptyset]$

$[(1, \text{bonjour}), (2, \text{allo}), \emptyset, \emptyset]$

$[(1, \text{bonjour}), (2, \text{allo}), \emptyset, (4, \text{hola})]$

$[(1, \text{bonjour}), (2, \text{allo}), (3, \text{salut}), (4, \text{hola})]$

Tri Comptage

Trier des entiers entre 1 et k en temps $O(n+k)$
avec répétitions possibles

$\forall i = 1..k, C[i] = \# \text{ de fois où } i \text{ apparaît dans l'entrée}$

Dans la sortie, mettre $C[1]$ fois 1, suivi de
 $C[2]$ fois 2, suivi de
 $C[3]$ fois 3, ...

$C[k]$ fois k

ex: $T = [8 \ 1 \ 7 \ 5 \ 8 \ 5]$

$C = [1 \ 0 \ 0 \ 0 \ 2 \ 0 \ 1 \ 2]$

ex: 1 - 2 0 1 7 3 8 3 1

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 2 & 0 & 1 & 2 \end{bmatrix}$$

1 2 3 4 5 6 7 8

$$S = [1 \ 5 \ 5 \ 7 \ 8 \ 8]$$

Tri Comptage (T)

$$O(n) \quad k = \max(T)$$

$$O(k) \quad C = [0] * k \quad // k \text{ fois } 0$$

$$O(n) \quad \left(\begin{array}{l} \text{pour } i = 1 \dots n \\ C[TE[i]] += 1 \end{array} \right)$$

Boucle B:

```

pour i = 1 .. k
    pour j = 1 .. C[i]
        S.append(i)
    
```

return S

$$T = [8 \ 1 \ 7 \ 5 \ 8 \ 5]$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 2 & 0 & 1 & 2 \end{bmatrix}$$

1 2 3 4 5 6 7 8

i=1 i=5 i=7 i=8

$$S = [1 \ 5 \ 5 \ 7 \ 8 \ 8]$$

Boucle B:

quand $i = 1: C + C[1] * c'$

incr. i incr. j
append

$$i = 2: c + C[2] * c'$$

$$i = k: c + C[k] * c'$$

Complexité: $O(nk)$ car la 2^e boucle domine la complexité et ne prendra plus qu'un temps $O(nk)$.

Complexité : $O(nk)$, car la boucle interne a complexité $O(k)$ et ne prendra plus qu'un temps $O(nk)$.

Pas faux, pourrait être mieux, l'algo n'est pas $\Omega(nk)$,
et donc pas $\Theta(nk)$.

Combien de temps la boucle B prend-elle au total?

Quand $i=1: c + C[1] \cdot c'$
 $i=2: c + C[2] \cdot c'$
...

$i=k: c + C[k] \cdot c'$

$$\text{Temps total : } \sum_{i=1}^k (c + C[i] \cdot c') = \sum_{i=1}^k c + \sum_{i=1}^k c' \cdot C[i]$$

$$= k \cdot c + c' \cdot \sum_{i=1}^k C[i]$$

$$= c \cdot k + c' \cdot n \in \Theta(k + n)$$

$$\text{où } k = \max(T)$$

• On peut ajuster pour atteindre $\Theta(n + (\max(T) - \min(T)))$

Version clé-valeur

Tri Comptage Clé Valeur (T) $T = [(3, a16), (2, h1), (3, h0a)]$

$k = \max(T.cles())$

$C[i] = [] * k$ // $[[], [], \dots []]$

pour $i = 1, \dots, n$
 $C[i].append(T[i])$ // ex: $C[3] = [(3, a16), (3, h0a)]$

$S = []$

pour $i = 1, \dots, k$
pour $j = 1, \dots, C[i].size()$

$S.append(C[i][j])$

$O(n+k)$

return S

Tri sur place

- Un algo de tri est sur place s'il utilise $O(1)$ de mémoire auxiliaire (i.e. on ne compte pas l'entrée), Donc pas de tableau temporaire.

• ... insertion ...

temporaire,

- sur place: insertion, quicksort wiki
- pas sur place: fusion, quicksort, merge, comptage
(A,B,S) (A,B) (M) (C,S)

Tri stable

Un algo de tri est stable si les elts égaux ont le même ordre relatif dans l'entrée et la sortie.

ex: $\left[\overset{(1)}{3} \overset{(1)}{2} \overset{(2)}{3} \overset{(1)}{1} \overset{(2)}{1} \overset{(3)}{3} \right]$

stable $\swarrow$ non-stable $\searrow$

$\left[\overset{(1)}{1} \overset{(2)}{1} \overset{(1)}{2} \overset{(1)}{3} \overset{(2)}{3} \overset{(3)}{3} \right]$ $\left[\overset{(2)}{1} \overset{(1)}{1} \overset{(1)}{2} \overset{(1)}{3} \overset{(3)}{3} \overset{(2)}{3} \right]$

- stable: insertion, fusion, comptage-clé-vel
- non-stable: quicksort